

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE
DO SUL**

ESCOLA POLITÉCNICA

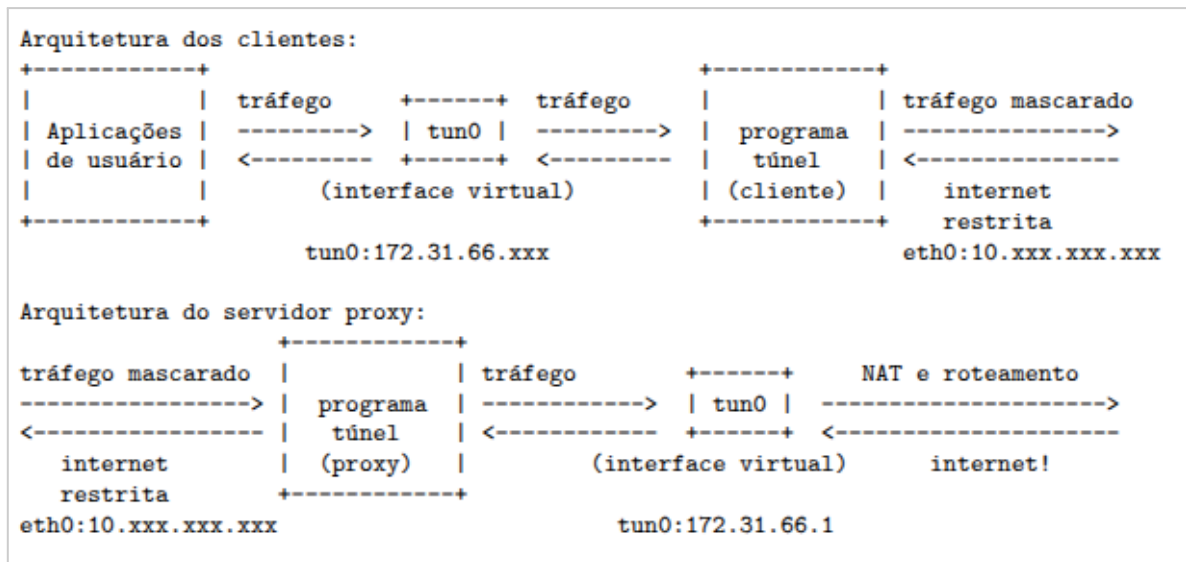
LABORATÓRIO DE REDES DE COMPUTADORES

**Monitor de Tráfego de Rede em Tempo Real utilizando Raw
Sockets**

Andrei Martins, Eduardo Cabral, Lara Kunrath Padilha, Leonardo Garcia Forquim Bertinatto

1. Introdução

O objetivo deste trabalho é implementar uma ferramenta para monitoramento de tráfego de uma rede em tempo real, utilizando raw sockets. Esta ferramenta deve capturar, interpretar e classificar os pacotes que trafegam em uma interface virtual que simula um túnel descrito no diagrama abaixo:



A aplicação desenvolvida é executada na máquina que atua como servidor (proxy), onde todo o tráfego da rede é concentrado. Para o monitoramento, a aplicação oferece uma interface de usuário em modo texto com estatísticas para cada cliente conectado e, ao mesmo tempo, gera arquivos de log separados em formato CSV para as diferentes camadas da pilha TCP/IP. Esses arquivos são atualizados em tempo real, o que permite acompanhar o tráfego durante a execução do programa.

2. Fundamentação Teórica

A pilha de protocolos TCP/IP é organizada em camadas, sendo as principais utilizadas neste trabalho: a camada de Internet (onde atuam protocolos como IPv4, IPv6 e ICMP), a camada de Transporte (TCP e UDP) e a camada de Aplicação (HTTP, DHCP, DNS, NTP, entre outros). Cada camada adiciona seus próprios cabeçalhos aos dados, possibilitando o roteamento, transporte confiável ou não confiável e a interpretação final pelas aplicações.

Os **raw sockets** permitem que uma aplicação tenha acesso direto aos pacotes na rede, incluindo os cabeçalhos das camadas inferiores. Diferentemente de um socket TCP ou UDP tradicional, que já entrega os dados “tratados”, um socket bruto fornece ao programa a possibilidade de inspecionar e interpretar manualmente os campos dos cabeçalhos de protocolos, como endereços IP, portas, flags, tipos de mensagem ICMP, etc. Por isso, esse tipo de socket é amplamente utilizado em ferramentas de monitoramento e diagnóstico de rede (por exemplo, sniffer de pacotes).

Além disso, o trabalho faz uso de uma **interface virtual de túnel (tun0)**. Esse tipo de interface é criado por um programa de túnel e funciona como se fosse uma placa de rede

lógica: o tráfego das aplicações dos clientes é encapsulado pelo túnel, enviado pela rede física e, no servidor, é reintroduzido na interface `tun0`, onde pode ser capturado pelo monitor.

Por fim, a escolha pelo formato **CSV** para os arquivos de log facilita a análise posterior dos dados, permitindo que os registros sejam abertos em planilhas ou processados por scripts para geração de gráficos e estatísticas.

3. Ambiente de Rede e Arquitetura

a. Arquitetura da rede

A rede utilizada no trabalho segue a arquitetura descrita no enunciado: um conjunto de clientes acessa a internet por meio de um servidor proxy, utilizando um programa de túnel. Nos clientes, o tráfego das aplicações de usuário passa por uma interface virtual (`tun0`), é encapsulado pelo programa de túnel e enviado pela interface física (`eth0`) até o proxy.

No servidor proxy, o programa de túnel recebe esse tráfego encapsulado, realiza o desencapsulamento e injeta os pacotes em uma interface virtual `tun0`. A partir dessa interface, uma regra de NAT e roteamento (usando `iptables`) encaminha os pacotes para a internet.

O monitor de tráfego foi desenvolvido para capturar pacotes justamente nessa interface `tun0` do servidor. Dessa forma, ele consegue enxergar o tráfego de todos os clientes, já desencapsulado, o que é ideal para a análise das camadas da pilha TCP/IP, mantendo a transparência em relação às aplicações de origem.

b. Ambiente de execução

Para a implementação e testes, foi utilizado o **container Linux** disponibilizado pela disciplina, que já contém as dependências necessárias (compilador, bibliotecas padrão e suporte a `iptables`).

Os testes foram realizados com três containers:

- 1 container atuando como **servidor proxy**, onde executam o programa de túnel (modo servidor) e o monitor de tráfego;
- 2 containers atuando como **clientes**, cada um com o túnel configurado em modo cliente, acessando a internet por meio do proxy.

Essa configuração simula uma LAN com múltiplas máquinas Linux conectadas à mesma rede, conforme sugerido no enunciado.

4. Projeto e Implementação da Solução

Instruções de uso

Todos os passos abaixo podem ser executados dentro do container disponibilizado para a disciplina.

- Compilação:
 - Entrar na pasta e executar: **make all**
- Execução:
 - Executar o arquivo *traffic_monitor* passando a interface alvo.
./traffic_monitor tun0

Como base para o projeto foi utilizado o container da disciplina para execução e testes. O projeto foi implementado em C atendendo as especificações e respeitando as limitações do container disponibilizado.

A implementação foi pensada de forma modular separando os requisitos em módulos:

capture.c: criação do socket raw e captura do pacote.

analyze.c: análise e classificação do pacote.

stats.c: exibição das estatísticas de na tela em tempo de execução.

log.c: escrita dos logs .csv de acordo com a classificação.

Todo o código implementado está comentando, abaixo seguem alguns pontos importantes para entendimento do fluxo principal.

main.c

```
while (running) {
    // 1. Captura o pacote
    int size = capture_packet(raw_socket_fd, packet_buffer);

    if (size > 0) {
        // 2. Processa e analisa (classifica) o pacote
        process_packet(packet_buffer, size, &packet_info);

        update_stats(&packet_info); // atualiza estatísticas

        // 3. Escreve nos arquivos de log (e faz a exibição em modo
        texto, omitida aqui)
        write_network_log(&packet_info);
        write_transport_log(&packet_info);
        write_application_log(&packet_info);
        packet_count_since_draw++;

    }

    //para evitar sobrecarga de recursos, escreve a cada 20 pacotes
    if (packet_count_since_draw >= 20) {
        draw_interface();
        packet_count_since_draw = 0;
    }
}
```

```
}
```

Criação do socket (capture.c):

setup_raw_socket(...)

```
// AF_PACKET para capturar na Camada de Enlace (inclui cabeçalho Ethernet)
// ETH_P_ALL para capturar todos os tipos de pacotes na Camada 2
int sock_fd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if (sock_fd < 0) {
    perror("Erro ao criar Raw Socket. Execute com sudo!");
    return -1;
}
```

```
// // 1. Vinculando a interface específica passada por parâmetro
if(setsockopt(sock_fd, SOL_SOCKET, SO_BINDTODEVICE, ifname,
(socklen_t)strlen(ifname)) < 0) {
    printf("SO_BINDTODEVICE(%s) falhou: %s", ifname,
strerror(errno));
    close(sock_fd);
    exit(EXIT_FAILURE);
}
```

Para simplificar foi criado uma estrutura (PacketInfo) que é utilizada após a classificação, está em protocols.h, nesta estrutura estão os dados extraídos utilizados para alimentar as estatísticas e os logs.

```
// Estrutura para agrupar todas as informações extraídas do pacote
typedef struct PacketInfo_sc{
    char timestamp[30];           // Data e hora da captura
    int total_len;                // Tamanho total do pacote em bytes

    // Camada de Rede (camada_internet.csv)
    char net_protocol[10];        // IPv4, IPv6, ICMP, outro
    char ip_src[INET6_ADDRSTRLEN]; // Endereço IP de origem
    char ip_dst[INET6_ADDRSTRLEN]; // Endereço IP de destino
    int next_proto_id;            // ID do protocolo carregado (TCP=6,
UDP=17, etc.)
    char icmp_info[100];          // Informações adicionais do ICMP

    // Camada de Transporte (camada_transporte.csv)
    char trans_protocol[10];       // TCP, UDP, outro
}
```

```

int port_src;           // Porta de origem
int port_dst;           // Porta de destino

// Camada de Aplicação (camada_aplicacao.csv)
char app_protocol[10];  // HTTP, DHCP, DNS, NTP, outro
char app_info[256];     // Informações do cabeçalho da
Aplicação
} PacketInfo;

```

5. Arquivos de Log

Foram implementados arquivos de log que permitem visualização enquanto a aplicação é executada.

a. Camada de Rede

O arquivo de log *camada_internet.csv* para o protocolo IP na camada de rede deve contemplar os protocolos IPv4, IPv6 e ICMP e ter as seguintes colunas:

- Data e hora que o pacote foi capturado;
- Nome do protocolo (IPv4, IPv6, ICMP, outro);
- Endereço IP de origem (ex: 100.114.7.75, fe80::ad3e:46fc:abf7:55c9);
- Endereço IP de destino;
- Número identificador do protocolo que está sendo carregado no pacote;
- Outras informações do protocolo (para ICMP);
- Tamanho total do pacote em bytes.

b. Camada de Transporte

O arquivo de log *camada_transporte.csv* para camada de transporte deve contemplar os protocolos TCP e UDP e ter as seguintes colunas:

- Data e hora que o pacote foi capturado;
- Nome do protocolo (TCP, UDP, outro);
- Endereço IP de origem;
- Porta de origem (ex: 8080);
- Endereço IP de destino;
- Porta de destino;
- Tamanho total do pacote em bytes.

c. Camada de Aplicação

O arquivo de log *camada_aplicacao.csv* para camada de aplicação deve contemplar os protocolos HTTP, DHCP, DNS e NTP e ter as seguintes colunas:

- Data e hora que o pacote foi capturado;
- Nome do protocolo (HTTP, DHCP, DNS, NTP, outro);
- Informações do protocolo (obtidas de seu cabeçalho).

6. Interface do Monitor de Tráfego

A interface do monitor é implementada em modo texto e atualizada periodicamente pela função *draw_interface()*. A cada 20 pacotes processados, a tela é redesenhada para evitar consumo excessivo de CPU e tornar a atualização visual mais estável.

De forma geral, a interface apresenta:

- **Contadores globais por protocolo**, mostrando quantos pacotes IPv4, IPv6, ICMP, TCP, UDP, HTTP, DNS, etc., foram capturados desde o início da execução;
- **Estatísticas por cliente**, agrupadas pelo IP da rede de túnel (endereços 172.31.66.x), indicando:
 - máquinas remotas mais acessadas;
 - portas e protocolos utilizados;
 - número de conexões e pacotes trocados;
 - volume aproximado de tráfego (em bytes).

Essas informações permitem, de forma rápida, identificar o perfil de tráfego de cada cliente, quais serviços externos estão sendo mais utilizados e se há algum comportamento que chame a atenção.

Foi implementada a seguinte interface que exibe as estatísticas dos dados monitorados.

```

===== Monitor de Tráfego Dual Stack (IPv4/IPv6) =====
Total Pacotes: 44200 | Volume Total: 21880913 bytes
Filtros: [172.31.66.*] e [fe80::*]

| CLIENTE (TUNEL) | TOTAL PKTS | TOTAL BYTES | TCP | UDP | ICMP | HTTP | DHCP | DNS | NTP | OUTROS |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| fe80::5fdc:ea7b:9fed:3653 | 1 | 48 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 172.31.66.101 | 1924 | 149816 | 1924 | 0 | 0 | 116 | 32 | 0 | 0 | 0 |
| fe80::bfd5:c0ee:bd26:7d92 | 6 | 492 | 0 | 0 | 6 | 0 | 0 | 0 | 0 | 0 |
| 172.31.66.102 | 1152 | 90672 | 1148 | 0 | 4 | 68 | 0 | 0 | 0 | 0 |
| fe80::bfee:4bf7:9027:658e | 6 | 492 | 0 | 0 | 6 | 0 | 0 | 0 | 0 | 0 |
  
```

Pressione CTRL+C para encerrar.

7. Testes e Resultados

Para os testes foram utilizados 3 containers, 1 como servidor e outros 2 como clientes. Para gerar o tráfego, foram abertos os navegadores de ambos os clientes com o fim de simular o que seria o uso de um equipamento normal.

8. Conclusão

O desenvolvimento do monitor de tráfego de rede em tempo real permitiu aplicar, de forma prática, os conceitos estudados ao longo da disciplina, especialmente no que diz respeito ao uso de raw sockets, análise de protocolos e organização modular de software. A ferramenta implementada foi capaz de capturar pacotes diretamente da interface de rede, classificá-los nas camadas de Internet, Transporte e Aplicação, além de registrar todas as informações relevantes em arquivos CSV separados, possibilitando visualização em tempo real.

A divisão do projeto em módulos facilitou a organização do código e a manutenção da aplicação, tornando o fluxo de captura, análise, exibição e registro de dados mais claro e eficiente. Os testes realizados com múltiplos contêineres mostraram que a solução funciona de maneira estável e atende aos requisitos propostos, exibindo estatísticas atualizadas e permitindo monitoramento contínuo do tráfego gerado pelos clientes.

Assim, o trabalho atingiu plenamente seus objetivos, resultando em uma ferramenta funcional, extensível e útil para compreensão prática do funcionamento dos protocolos de rede e do monitoramento de tráfego em sistemas reais.